Efficient Implementation and Evaluation of BM25 and BM25F Algorithms

Author: Balseit Yeldana
Faculty of Information Technology

---

1. Topic Proposal and Project Goals

The goal of this project is to design, implement, and rigorously evaluate efficient implementations of the classical information retrieval algorithms **BM25** and **BM25F**, with a particular focus on execution speed and retrieval effectiveness. Although BM25 and its field-aware extension BM25F are well-established methods in information retrieval, their practical performance strongly depends on implementation details, indexing strategies, and parameter tuning.

The project has three main objectives.

First, to implement BM25 and BM25F *from scratch* using efficient data structures and vectorized computation, and to compare the resulting system against reference implementations.

Second, to analyze performance bottlenecks and demonstrate concrete optimizations that significantly reduce query latency without sacrificing retrieval quality.

Third, to investigate whether LLM-generated document fields (such as enhanced titles or keywords) can improve retrieval performance when integrated into BM25F, and under what conditions such improvements are achievable.

The project deliberately avoids neural retrieval models and focuses on classical probabilistic retrieval to highlight how far traditional methods can be pushed through careful engineering and tuning. The final outcome is a clear comparison between baseline BM25, manually weighted BM25F, LLM-enhanced BM25F, and tuned BM25F variants optimized via grid search and Hyperopt.

---

2. Problem Description and Semi-Formal Definition

Let a document collection $D = d_1, d_2, \dots, d_N$ and a query set $Q = \{q_1, q_2, \dots, q_M\}$ be given. Each document d is represented as a sequence of terms drawn from a vocabulary V. The retrieval task is to rank documents $d \in D$ for a given query $q \in Q$ such that documents relevant to the query appear as high as possible in the ranking.

In probabilistic information retrieval, this ranking is typically achieved by estimating a relevance score $\text{score}(q, d)$ that approximates the probability of relevance $P(R = 1 \mid q, d)$. BM25 is a widely used approximation of this probability based on term frequency statistics and document length normalization.

In the BM25F setting, each document d is decomposed into a set of fields $f \in F$, such as title, abstract, or body text. The retrieval problem then becomes computing a relevance score that aggregates contributions from multiple fields while accounting for their differing importance.

Formally, the problem addressed in this project is:

Given a fixed document collection and query set, design an efficient BM25/BM25F-based retrieval system that maximizes standard IR evaluation metrics $\boldsymbol{MAP}, \boldsymbol{Precision@k}, \boldsymbol{Recall@k}$ under practical constraints on computation time and memory usage.

---

3. Semi-Formal Definition of the Implemented Algorithms

3.1 BM25

For a query q consisting of terms t, the BM25 score of a document d is defined as:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{tf(t,d) \cdot (k_1 + 1)}{tf(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)})\}$$

where:

- $tf(t,d)$ is the term frequency of term t in document d,
- $|d|$ is the document length,
- avgdl is the average document length in the collection,
- $k_1 > 0$ and $b \in [0,1]$ are hyperparameters,
- $\mathrm{IDF}(t)$ is the inverse document frequency, computed globally.

3.2 BM25F

In BM25F, each document is represented as a set of fields $f \in F$. For a term t, the field-normalized term frequency is:

$$\widetilde{tf}(t,d) = \sum_{f \in F} w_f \cdot \frac{tf(t,d_f)}{1 - b_f + b_f \cdot \frac{|d_f|}{\mathrm{avgdl}_f}}$$

where:

- $tf(t,d_f)$ is the frequency of term t in field f,
- w_f is the weight assigned to field f,
- b_f is the field-specific normalization parameter.

The BM25F score is then:

$$\mathrm{BM25F}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{\widetilde{tf}(t,d) \cdot (k_1 + 1)}{\widetilde{tf}(t,d) + k_1}\}$$

In this project, a simplified BM25F variant is used, where a single global b parameter is shared across fields, and only field weights w_f are tuned.

---

4. Implementation, Experimental Setup, and Datasets

4.1 Implementation Details

All algorithms were implemented in **Python** using **NumPy** and **SciPy**.

BM25Okapi from rank-bm25(0.2.2) was used as a reference implementation for comparison.

The BM25F implementation extends the BM25 pipeline by maintaining separate sparse matrices per field and combining them at query time using weighted aggregation.

LLM-based field extraction was performed using **Phi-3-mini-4k-instruct**, applied offline to generate enhanced titles, keywords, and additional metadata fields.

4.2 Experimental Setup

- **Dataset:** SCIDOCS
- **Corpus size:** 25,657 documents
- **Vocabulary size:** 102,376 unique terms
- **Queries:** 1,000
- **Average relevant documents per query:** 29.9
- **Density:** 0.0068–0.073%

Only the official query relevance judgments were used. No training was performed; all tuning was conducted on a validation subset.

Dataset Examples:

Query: {'_id': '78495383450e02c5fe817e408726134b3084905d', 'text': 'A Direct Search Method to solve Economic Dispatch Problem with Valve-Point Effect'}

Relevance: {'query-id': '78495383450e02c5fe817e408726134b3084905d', 'corpus-id': '632589828c8b9fca2c3a59e97451fde8fa7d188d', 'score': 1.0}

Document:

- _id: 632589828c8b9fca2c3a59e97451fde8fa7d188d

- title: A hybrid of genetic algorithm and particle swarm optimization for recurrent network design
- text: An evolutionary recurrent network which automates the design of recurrent neural/fuzzy networks using a new evolutionary...

4.3 Hyperparameters and Bounds

BM25 Parameters:

- k_1: [1.2, 2.5], default 1.5
- b: [0.3, 0.9], default 0.75

Field Weights (LLM-enhanced):

- Enhanced title: [0.0, 6.0]
- Keywords: [0.0, 4.0]
- Abstract: [0.0, 3.0]
- Original text: [0.0, 4.0]

4.4 Hardware and Computation

- CPU-based execution for retrieval experiments
- GPU (Tesla T4, 15.8 GB) used only for LLM-based document field generation
- Query evaluation time ranged from ~6 ms (optimized BM25) to ~800 ms (naive BM25F)

4.5 Contribution vs External Sources

- **Original work:** BM25/BM25F implementations, optimizations, evaluation framework, grid search strategy, and analysis
- **External sources:** BM25/BM25F formulas from Robertson & Zaragoza (2009); LLM model from Microsoft

---

5. Evaluation and Analysis of Experiments

5.1 Implementation Correctness

This section presents a step-by-step evaluation of the implemented BM25 and BM25F algorithms, following the chronological order of the experiments: correctness verification, efficiency optimization, pseudo-relevance feedback, hyperparameter optimization, and LLM-based document enhancement.

The first experiment aimed to verify the correctness of the custom BM25 implementation by comparing it against a widely used reference implementation, BM25Okapi from the rank_bm25 library.

Model	MAP	P@5	P@10	R@20	R@100	Time (ms)
MyBM25	0.0171	0.1078	0.0769	0.0345	0.0596	476.0
MyBM25F	0.0168	0.1076	0.0775	0.0339	0.0576	805.8

Reference_BM25	0.0171	0.1072	0.0765	0.0346	0.0596	52.6

The MAP score of MyBM25 exactly matches the reference implementation (0.0171), with zero numerical difference. Minor variations in precision and recall values are within floating-point tolerance and do not affect ranking behavior.

Conclusion:
This confirms that the implemented BM25 scoring function is functionally correct and consistent with the standard formulation.

5.2 BM25F Effectiveness (Unoptimized)

The second experiment evaluated whether BM25F provides an advantage over standard BM25 on the given dataset when using only two fields (title and text).

- BM25 MAP: 0.0171
- BM25F MAP: 0.0169
- Relative change: −1.36%

BM25F did not improve retrieval quality in this configuration and produced nearly identical results to BM25.

Conclusion:
On this dataset, field separation alone does not provide a measurable benefit without careful weighting or additional semantic information.

5.3 Efficiency Analysis and Speed Optimization

The third experiment focused on performance optimization. Initially, the unoptimized BM25F implementation was significantly slower due to nested loops and repeated computations.
The main optimizations include:

- Sparse term-document matrices stored in CSR (Compressed Sparse Row) format.
- Pre-computation of document length normalization factors.
- Vectorized scoring using NumPy operations instead of per-document loops.
- Global IDF computation with Boolean OR document frequency counting.
- Memory-efficient indexing of term positions and frequencies.

Implementation	Query Time	Speed vs Reference
Initial BM25F	805.8 ms	15.3× slower
Optimized BM25F	14.4 ms	3.65× faster
Reference BM25	52.6 ms	—

5.4 Pseudo-Relevance Feedback (PRF)

Pseudo-relevance feedback was evaluated using two strategies:

- **RM3 expansion**, using the top 3 retrieved documents,
- **Conservative expansion**, adding a small number of high-IDF terms.

The RM3 configuration used parameters:

- $\alpha = 0.8$, $\beta = 0.2$,
- top_k_feedback = 3,
- IDF thresholds to suppress noisy terms.

Observed effects:

- Aggressive expansion (15 terms): MAP decreased by 7%
- Conservative expansion (2 terms): MAP decreased by 2.4%
- Recall@100 increased slightly (+4.9%)
- Even tuned PRF underperformed baseline BM25

Conclusion:
PRF improved recall but consistently harmed precision, leading to lower MAP. This indicates that term drift outweighed relevance gains for this dataset.

5.5 Hyperparameter Optimization with Hyperopt

Hyperparameter optimization was then applied to the optimized BM25F (two-field setup: title and text).

Setup:

- 700 training queries, 300 validation queries
- 50 trials
- Tree-structured Parzen Estimator (TPE)
- Search ranges:
  - title weight: [0.5, 4.0]
  - text weight: [0.5, 2.5]
  - k1: [1.2, 2.0]
  - b: [0.6, 0.95]

Results:

- MAP improved from 0.0171 → 0.0173 (+0.9%)
- P@10 improved from 0.0769 → 0.0789 (+2.6%)

The improvement was measurable but small.

Conclusion:
Hyperopt provided limited gains and did not significantly alter retrieval quality.

5.6 LLM-Enhanced BM25F

In the final stage, an LLM (Phi-3-mini-4k-instruct) was used to decompose documents into structured fields:

- enhanced title,
- keywords,
- abstract,
- methodology,
- findings,
- contribution.

Experiments were conducted on a subset of 200 documents, where LLM inference was feasible.

Mean Average Precision Comparison

Model and Method	MAP	Improvement
BM25 (Baseline)	0.1541	−
BM25F (Manual weighting)	0.1486	−3.6%
BM25F-LLM (HyperOpt)	0.1072	−30.4%
BM25F-LLM (Grid Search)	0.1591	+3.2%

Baseline BM25 achieved a MAP of 0.1541. A manually weighted BM25F variant underperformed this baseline, indicating that naive field weighting is insufficient. Introducing LLM-generated fields without tuning further degraded performance, despite improving recall.

Hyperparameter optimization using **Hyperopt (TPE)** failed to identify good configurations, resulting in a MAP drop of over 30%. This is attributed to the small, irregular search space and the sensitivity of BM25F to weight misconfiguration.

In contrast, **grid search** over a carefully designed parameter grid achieved a MAP of 0.1591, corresponding to a +3.2% improvement over baseline BM25. The optimal configuration emphasized enhanced titles(5.0), keywords(2.5), original text(3.0), abstract(1.0) while assigning zero weight to low-quality LLM-generated fields.

Precision–recall analysis showed that LLM-enhanced BM25F primarily improved recall, while precision gains required careful tuning. Query-time benchmarks demonstrated that optimized implementations achieved up to an order-of-magnitude speedup compared to naive baselines.

---

6. Discussion and Conclusions

This project demonstrates that classical retrieval models can be substantially improved in efficiency and modestly improved in effectiveness, provided that implementation and tuning are handled carefully.

The first major result is the successful implementation of BM25 and BM25F from scratch, with correctness verified against a reference system. The optimized implementation achieved a 447× speedup over the initial version and a 29× speedup over the reference, highlighting the importance of specialization, vectorization, and sparse data structures.

BM25F, when applied naively, did not outperform BM25. Similarly, pseudo-relevance feedback degraded MAP despite improving recall, showing that query expansion can be harmful in sparse or noisy relevance settings.

Hyperparameter optimization using Hyperopt provided only marginal gains and, in some cases, significantly worsened performance. In contrast, grid search proved more reliable, as it systematically explored a small parameter space without relying on probabilistic assumptions.

LLM-based document enhancement showed mixed results. Untuned LLM-enhanced BM25F performed worse than baseline BM25. However, after careful grid search tuning, LLM-enhanced BM25F achieved a +3.2% MAP improvement, confirming that LLMs can be beneficial only when extraction quality and weighting are controlled.

Overall, the results show that:

- Optimization and implementation choices matter more than model complexity,
- LLMs are not a guaranteed improvement for classical retrieval,
- Careful tuning is essential for any retrieval enhancement.

---

7. References

[1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval.